05

React no Ecossistema Laravel

Inertia + React na prática

Interfaces modernas com TypeScript e limites claros entre UI e domínio.

Roteiro da aula

- Problema real na agenda
- O papel do React
- Inertia + React
- Estrutura por feature
- Componentes e estado
- Formulários e validação
- Dados remotos e mutations
- Projeto e checklist

Problema real: agenda reativa

- A recepção precisa filtrar a agenda sem recarregar a página.
- Filtros por data, profissional e status respondem imediatamente.
- O estado permanece previsível.
- Erros de validação do Laravel aparecem no formulário.
- As regras de negócio continuam no backend.
-

O papel do React

React funciona bem em interfaces com muitos estados e interações.
Componentes reutilizáveis reduzem repetição.
TypeScript explicita contratos de props.
O frontend oferece feedback imediato.
Laravel continua responsável por autenticação, autorização, validação e regras de negócio.

Inertia + React

Definição

Inertia conecta rotas Laravel a páginas React sem exigir uma API separada.

Bom para

Navegação com experiência de SPA.

Autenticação e rotas no Laravel.

Props vindas do controller.

Validação compartilhada com o backend.

Exemplo de código

```
return Inertia::render('Appointments/Index', [
    'appointments' => AppointmentResource::collection(
        Appointment::query()->forDate($date)->get()
    ),
]);
```

Estrutura por feature

Objetivo

Manter página, componentes, hooks e contratos próximos da mesma capacidade do produto.

Resultado

Navegação simples no código.

Dependências mais claras.

Testes concentrados por feature.

Menos pastas genéricas globais.

Exemplo de código

```
resources/js/
  Pages/Appointments/Index.tsx
  Features/Appointments/
    components/
    hooks/
    schemas/
    api/
```

Componentes e estado

Responsabilidades
A Page recebe props tipadas do Laravel.
Componentes de feature cuidam da interação e da apresentação.
Hooks conectam Inertia, formulários e APIs do navegador.

Estado
Mantenha estado de UI perto de quem o usa. Calcule valores derivados durante a renderização e evite copiar props para outro estado.

Formulários e dados remotos

- Use useForm para enviar dados às rotas Laravel.
- Mostre no formulário os erros de validação do backend.
- Use props Inertia para os dados iniciais e fluxos simples.
- Adote TanStack Query quando a tela exigir cache, refetch, polling ou mutations independentes.
- Mantenha uma única fonte para cada dado.
-

Projeto do módulo

- Refatorar a agenda para Inertia + React sem mover regras de negócio para o navegador.
- Criar Pages/Appointments/Index.tsx.
- Tipar as props da página.
- Extrair AppointmentFilters.
- Enviar o formulário com useForm.
- Exibir erros retornados pelo Laravel.
- Testar o fluxo crítico no backend e na interface.
-

Checklist final

- Escolher React quando a interatividade justificar a complexidade.
- Criar uma página Inertia com props tipadas.
- Separar Page, componentes de feature e regras do domínio.
- Organizar estado local, formulário e dados remotos sem duplicação.
- Tratar no React os erros de validação do Laravel.
- Reconhecer quando TanStack Query acrescenta valor.
-